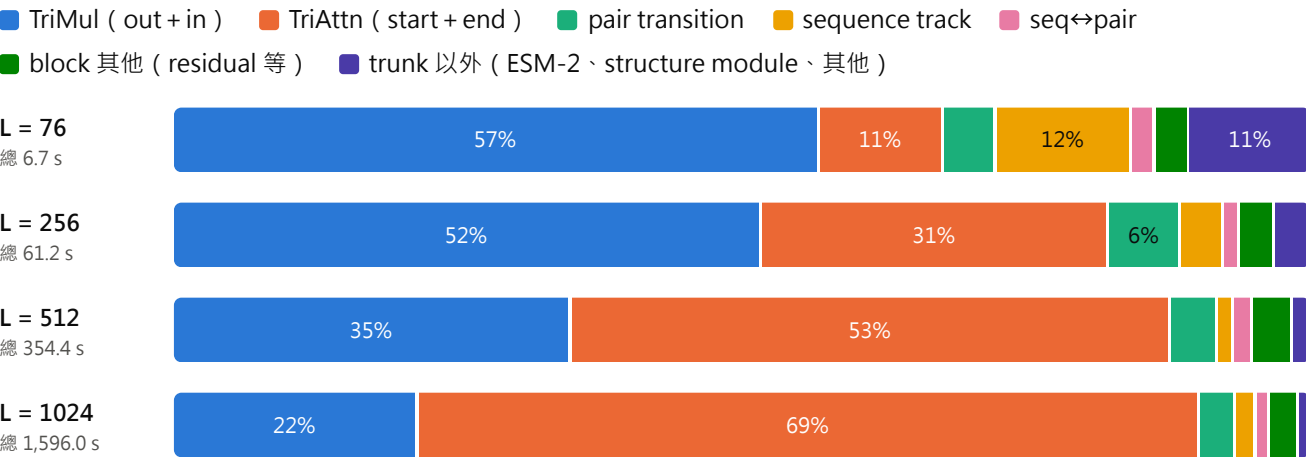


ESMFold 時間分布與結論 (workload characterization)

2026-10-06 · 學校 server CPU (無 GPU) · 完整推論流程 · FP32、不分塊 · 標示：實測 量到的 推導 由實測數字計算 假設 加上的前提、尚未驗證

1. 整條流程的時間佔比隨 L 的變化



一條橫條 = 一次完整推論 (從胺基酸序列到 3D 結構) 的總時間，切成互不重疊的 7 組；長度依序為 76 (泛素)、256、512、1024。各組包含的細項與秒數見表 1。

一眼看出的三件事：(1) 三角運算 (藍 + 橘) 隨 L 越佔越多，L = 1024 達 90%；(2) TriMul (藍) 越來越小、TriAttn (橘) 越來越大，在 L = 256 ~ 512 之間交叉；(3) ESM-2 與 structure module (紫) 在 L ≥ 256 時幾乎看不到。

表 1 全部類別的時間與佔比 (192 次 block 呼叫的總和)

類別	L = 76		L = 256		L = 512		L = 1024	
	秒	%	秒	%	秒	%	秒	%
Triangle Multiplication (outgoing)	1.92	28.7	16.36	26.7	62.12	17.5	171.89	10.8
Triangle Multiplication (incoming)	1.89	28.2	15.31	25.0	61.95	17.5	172.11	10.8
Triangle Attention (starting node)	0.37	5.5	9.38	15.3	93.14	26.3	487.88	30.6
Triangle Attention (ending node)	0.36	5.4	9.33	15.2	93.82	26.5	609.17	38.2
pair transition (mlp_pair)	0.31	4.6	3.87	6.3	14.72	4.2	51.05	3.2
sequence attention	0.40	6.0	1.53	2.5	3.75	1.1	18.69	1.2
sequence transition (mlp_seq)	0.39	5.8	0.78	1.3	1.22	0.3	10.12	0.6
LayerNorm (sequence)	0.01	0.1	0.02	0.0	0.04	0.0	0.05	0.0
sequence → pair	0.11	1.6	0.64	1.0	4.20	1.2	11.85	0.7
pair → sequence	0.03	0.4	0.22	0.4	1.73	0.5	6.32	0.4
block 其他 (residual 加法等)	0.20	3.0	1.88	3.1	12.43	3.5	41.07	2.6
ESM-2 (語言模型)	0.52	7.8	1.18	1.9	2.88	0.8	6.40	0.4
structure module	0.19	2.8	0.61	1.0	2.06	0.6	8.03	0.5
其他 (embedding、recycling 等)	0.02	0.3	0.07	0.1	0.39	0.1	1.36	0.1
總時間	6.7	100	61.2	100	354.4	100	1,596.0	100
峰值記憶體 (整個程式)	19.9 GB		22.2 GB		27.4 GB		60.7 GB	

2. 結論

結論 1 整條流程的瓶頸是 pair 表示上的三角運算

- $L \geq 256$ 時，TriMul + TriAttn 佔整條流程 82 ~ 90% ; 48 個 block 合計 97 ~ 99% 。 實測
- ESM-2 + structure module : $L \geq 256$ 時不到 3% 。 ESM-2 佔全部參數的 80% ，卻只佔不到 1% 的時間；參數量決定權重佔多少記憶體，時間則由中間資料 (pair 表示 L^2 、三角運算 L^3) 決定。 實測
- 筆電 GPU 的端到端量測一致 ($L = 512$ 時 block 佔 98.3%) ；與 LightNobel 只加速 trunk 的選擇一致。 實測

結論 2 瓶頸隨長度轉移；趨勢跨平台一致，交叉點與硬體有關

- TriMul 佔比 57% $\rightarrow$ 52% $\rightarrow$ 35% $\rightarrow$ 22% ; TriAttn 11% $\rightarrow$ 31% $\rightarrow$ 53% $\rightarrow$ 69% ($L = 76 \rightarrow 1024$) 。 實測
- 交叉點：server CPU 在 $L = 256 \sim 512$ 之間；筆電 GPU 在 $L < 256$ ；論文 H100 的趨勢相同 (TriAttn 29% $\rightarrow$ 76%) 。 實測
- 原因：兩者都有 L^3 運算，但 TriMul 的資料只有 L^2 (每筆資料被重複使用 L 次) ，TriAttn 會產生 L^3 大小的 logits ($L = 1024$ 時 17 GB) ，搬運量跟著 L^3 成長。 推導
- 跨平台成立的是趨勢，不是比例；FPGA 上的比例由自己的設計決定。 推導

結論 3 TriAttn 受記憶體頻寬限制，已貼近天花板 $\rightarrow$ 要減少搬運量

- tri_att_start ($L = 1024$) : add_ 31% + softmax 28% = 59% 在讀寫 logits ；矩陣乘法只佔 27% 。 實測
- add_ 的名目頻寬約 112 GB/s (計數的 68.7 GB $\div$ 實測 0.616 s) ，機器上限約 100 GB/s (同種運算、同樣的記憶體擺放實測) 。 推導
- roofline 達成率 58 ~ 88% 。 $L = 1024$ 峰值記憶體 60.7 GB ，主要是不分塊的 logits 。 推導
- softmax 的依賴只需要「一列」 ($L = 1024$ 時 4 KB) ，17 GB 的搬運來自「一個運算處理完整個張量」的執行方式 $\rightarrow$ 一列一列算、logits 不寫回記憶體 (FlashAttention 式、LightNobel 的 token-wise attention) 。分塊只降容量、不降搬運量 (GPU 實測時間不變) 。 推導

結論 4 TriMul 的時間大半浪費在資料排列，不在計算

- copy_ (permute 後的複製) 佔 TriMul 的 50 ~ 58% (CPU) ，GPU 上 57% ；einsum 本身只佔 14 ~ 17% (CPU) 、6% (GPU) 。 實測
- roofline 達成率只有 16 ~ 24% ，三種運算中最低。 推導
- permute 複製只有 9 GB/s ，連續複製 76 ~ 82 GB/s ，慢約 9 倍：同一 channel 的相鄰元素相隔 512 bytes ，每抓一條 64 bytes 的 cache line 只用到 4 bytes 。 實測
- 對照實驗：同形狀 (128 個 $1024 \times 1024 \times 1024$) 、資料已排好的矩陣乘法只要 63 ms (等於機器上限) ；HF 的寫法是 einsum 149 ms + copy 436 ms = 585 ms ，差 9.3 倍。形狀本身沒有問題，損失全在排列。 實測
- 根源：bmm 要求 [c][i][k] ，資料原本是 [i][k][c] 。a、b 各轉一次、結果轉回一次，共 3 次 (GPU 上也是 3 個 copy kernel) 。 推導

結論 5 只修好 TriMul 的排列，整條流程可省約 10 ~ 13% (L = 1024) 、 19 ~ 24% (L = 512)

- 一次 tri_mul_out (L = 1024 · 881 ms) = A 組 (einsum + copy) 585 ms + B 組 (linear 、 gate 、 LayerNorm) 296 ms 。 實測
- 保守情境 (省掉進入前的 2 次 copy ，結果仍要轉回 1 次) : A 組 63 + 124 = 187 ms → TriMul 快 1.82 倍 → 整體省 9.7% 。 推導 假設
- 理想情境 (3 次 copy 全省) : A 組 63 ms → TriMul 快 2.45 倍 → 整體省 12.8% 。 推導 假設
- L = 512 : 理想 einsum 未實測，以效率範圍估算，整體省 19 ~ 24% 。 推導 假設
- GPU 數據獨立估算 : TriMul 26.1% × copy 57.2% ≈ 整體省 15% ，與 CPU 一致 。 推導
- 假設 B 組不變；若再把 B 組融合 (中間結果不寫回) ，還能更省 → 以上為保守估計。只優化 TriMul 的上限 : L = 512 最多 1.54 倍、L = 1024 最多 1.28 倍 。 推導

結論 6 資料排列的問題不只出現在 TriMul

- L = 1024 時 tri_att_end 比 start 慢 583 ms ，其中 578 ms (99%) 來自 add_ (1,194 vs 616 ms) ； L = 512 時兩者相同 。 實測
- end 先把 pair 張量轉置，算出的 bias 也是轉置後的排列，加到 logits 時跳著讀取 。 推導

對 FPGA 設計的意義

- 本專題的 kernel 保留原始的 [i][k][c] 、直接寫出 z[i][j][c] ， 3 次 copy 全部不需要 (相當於結論 5 的理想情境；CPU / GPU 受函式庫格式限制，通常只能做到保守情境) 。 512-bit 讀取 = 16 個連續 channel ，直接對應 16 條 lane 。 推導
- 下一步：融合 LayerNorm 、 linear 、 gate 與 einsum ，消除 B 組的搬運；同樣的策略可推廣到 TriAttn (一系列算、logits 留在晶片上) 。 推導
- 以上只說明 FPGA 上不需要 copy ，不代表 FPGA 版一定比 CPU 快；實際速度取決於乘法器數量與記憶體頻寬，需另外評估。

3. 方法摘要

- 平台：學校 server，2 × Intel Xeon Gold 6526Y (共 32 實體核)，125 GB RAM，無 GPU；Python 3.9、torch 2.8.0+cpu、transformers 4.57.6。量測時負載為 0。
- 完整流程 (run_cpu.py)：HF ESMFold 全部 FP32 放 CPU、不分塊、32 執行緒，每個模組前後掐碼錶；類別互不重疊，加總等於總時間。48 個 block × 4 輪 recycling = 192 次呼叫 (探針實測)。
- 正確性：泛素預測結構與 PDB 1UBQ 的 CA RMSD = 0.83 Å (筆電 GPU 版 0.84 Å)。
- 拆解 (diagnose_cpu.py)：單一 block 的子模組，暖機後以 profiler 量 3 次；與完整流程的每次時間差 1.6% (TriMul)。
- 天花板 (ceilings_cpu.py)：矩陣乘法約 4,300 GFLOP/s ($n = 16384$ ， $\pm 5\%$)；頻寬約 100 GB/s (原地加法，記憶體分散在兩顆 CPU，與 ESMFold 的大張量相同；集中在一顆時只有一半)。
- 運算量、搬運量：count_ops.py 以 meta tensor 計數；搬運量是「每個運算讀所有輸入、寫所有輸出」的名目值，不含快取效應，也不含 CPU 內部的逐 channel 複製。

4. 限制

- 每個長度只跑一次；天花板同一點重複量測約 $\pm 5\%$ 。
- $L = 512$ 的理想 einsum 未實測 (以範圍估算)；「9.3 倍」對照實驗只在 CPU 上做過。
- $L > 1024$ 只能依趨勢外推。只量推論、batch = 1。
- 單獨量的 TriAttn 比完整流程裡快 22 ~ 39%，原因只解釋一部分 (新配置記憶體慢約 22%)；佔比以完整流程為準。
- 峰值記憶體是整個程式的最大值 (含載入模型)，短序列的數字主要反映載入時的用量。